I2C Communication Protocol

Digital IC Design Using FPGA Course Project

National Technology Institute (NTI)

Team Members	Code
Karim Yasser Mohamed Ali	313907
Mohamed Hany Sameh Ibarahim	240172
Marco Mina Sadek	233910

Table of Contents

1. Introduction

Modern digital systems often require multiple integrated circuits (ICs) to exchange information efficiently. Using parallel communication for this purpose requires a large number of interconnecting wires, increasing hardware complexity and cost. Serial communication protocols overcome this limitation by allowing devices to communicate reliably using only a few signal lines.

This project presents the design and implementation of an Inter-Integrated Circuit (I²C) communication system using Verilog HDL. The system consists of an I²C Master and an I²C Slave communicating over a shared two-wire bus. The master is responsible for initiating and controlling all communication, while the slave monitors the bus and responds only when it is addressed by the master.

The objective of this project is to design and implement a fully functional I²C communication system, including both the Master and Slave modules, and to verify its operation using a self-checking testbench that covers the possible communication scenarios. This report first explains the operation of the I²C protocol. It then presents a detailed breakdown of the RTL implementation for each module and discusses the verification methodology used to validate the design.

2. Historical Background

The Inter-Integrated Circuit (I²C) protocol was developed by Philips Semiconductors (now NXP Semiconductors) in 1982. At that time, electronic systems were becoming increasingly complex, requiring multiple integrated circuits to communicate with one another.

Connecting every device using parallel communication required many wires, increasing hardware complexity, cost, and PCB area. I²C was introduced as a simple serial communication protocol capable of connecting multiple devices using only two communication lines regardless of the number of devices on the bus.

I²C became one of the most widely used communication protocols in embedded systems. It is commonly found in microcontrollers, EEPROM memories, temperature sensors, real time clocks, LCD modules, ADCs, DACs, and numerous digital peripherals.

3. Project Overview

The objective of this project is to design and implement a complete I²C communication system consisting of a single I²C Master connected to a shared two-wire bus. Although the I²C protocol supports communication with multiple slave devices on the same bus, this implementation includes a single I²C Slave for verification purposes.

The master controls the entire communication process by generating the serial clock (SCL), issuing the START and STOP conditions, transmitting the slave address, and managing all read and write transactions. The slave continuously monitors the shared bus and only responds when its assigned address is detected.

The system communicates over two shared bus lines:

- SCL: Serial Clock generated by the master.
- SDA: Bidirectional Serial Data line shared by all devices.

The implemented design supports two communication modes:

- Write Operation: Master → Slave
- Read Operation: Slave → Master

Each transaction transfers a single 8-bit data byte between the master and the addressed slave, with acknowledgement (ACK) used after both the address and data bytes to ensure reliable communication.

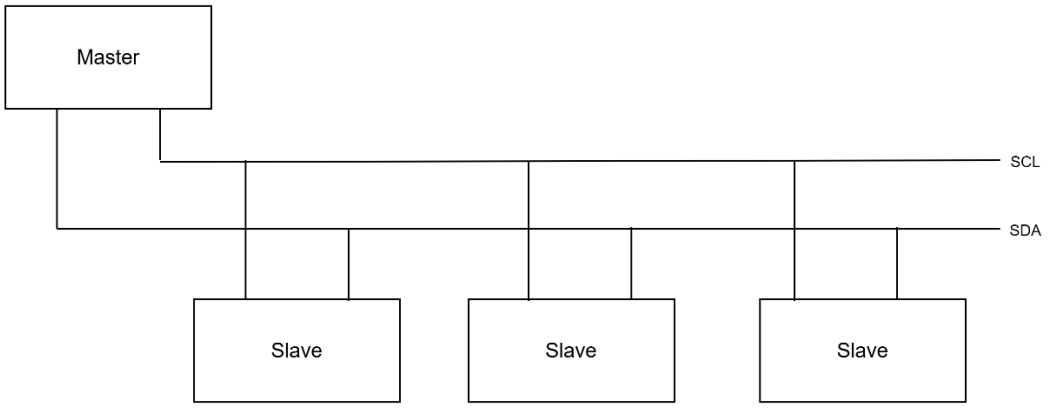


Figure 1- I²C General Structure

4. I²C Bus Signals

The I²C protocol uses only two communication lines:

4.1 Serial Clock (SCL)

The Serial Clock line synchronizes communication between the master and the slave. Unlike many digital communication systems where each device operates using its own clock, the I²C protocol relies on a single clock generated exclusively by the master.

In this implementation, the master generates the SCL signal using a clock divider module that divides the 50 MHz FPGA system clock to the desired 100 kHz I²C clock frequency.

The slave does not generate a clock; instead, it continuously monitors the SCL line and performs its operations according to its rising and falling edges, ensuring that both devices remain synchronized throughout the communication process.

4.2 Serial Data (SDA)

The SDA line carries all data and transmits it serially.

The transmitted sequence consists of:

- START condition
- 7-bit slave address
- Read/Write bit
- ACK bit
- Data byte
- ACK bit
- STOP condition

These components are transmitted in a fixed order, both devices always know what each received bit represents.

5. Open-Drain Communication

Neither the master nor the slave can actively drive SDA high.

Instead, each device can only:

- Drive SDA Low
- Release SDA Line (High Impedance Z)

When neither device drives SDA low, an external pull-up resistor automatically pulls the line high. Therefore:

Derive SDA Low represents a logic 0.

Release SDA represents a logic 1.

This arrangement prevents electrical conflicts when multiple devices share the same communication line.

6. Clock Divider

The FPGA system clock (50 MHz) operates at a much higher frequency than the desired I²C communication speed (100 kHz).

The master therefore contains a clock divider.

The clock divider counts system clock cycles and toggles the SCL output after a predefined number of counts, effectively generating the lower-frequency communication clock.

The slave does not require a clock divider because it receives the SCL signal generated by the master.

7. Communication Sequence

Every I²C transaction follows the same general sequence.

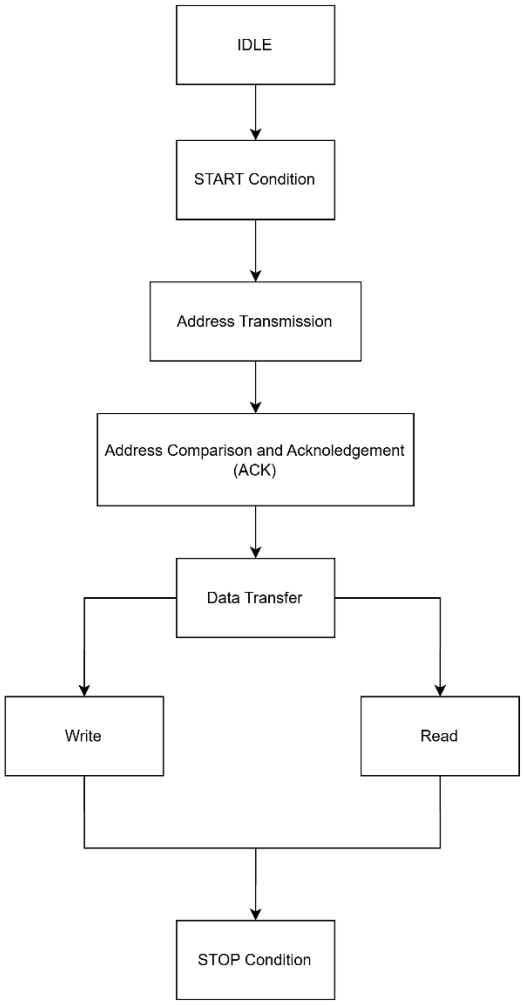


Figure 2- Block diagram for the communication sequence between the Master and the Slave.

7.1 Idle

Initially,

SCL and SDA remain High.

Neither device is transmitting data.

The slave continuously monitors the communication bus while waiting for a START condition.

7.2 START Condition

The master initiates communication by forcing SDA from High to Low while SCL remains High.

This transition informs every slave that a communication transaction has begun.

7.3 Address Transmission

After the START condition, the master transmits the seven-bit slave address followed by the Read/Write bit.

Each bit is transmitted serially over the SDA line.

The slave receives every bit and reconstructs the complete address.

7.4 Address Comparison and Acknowledgement (ACK)

The slave compares the received address with its own predefined address.

If the addresses do not match, the slave ignores the transaction until a STOP condition occurs.

If the addresses match, the slave acknowledges the transmission by pulling SDA low during the ninth clock pulse, this generates the ACK bit.

The master samples SDA during this clock pulse.

If SDA is low, communication continues.

If SDA remains high, the master interprets this as a Not Acknowledge (NACK) and terminates the communication

7.5 Data Transfer

The communication now proceeds according to the Read/Write bit.

7.5.1 Write Operation

The master transmits one byte of data serially.

The slave samples each bit and reconstructs the complete byte.

After receiving all eight bits, the slave sends another ACK.

The received byte is then stored inside the slave.

7.5.2 Read Operation

Similarly to the write operation, but the roles are reversed. The Slave becomes the transmitter and the Master becomes the receiver.

The slave loads its stored byte into a shift register.

The slave then transmits one bit during every clock cycle until all eight bits have been sent.

The master samples each bit and reconstructs the transmitted byte.

7.6 STOP Condition

Finally, the master ends communication by allowing SDA to transition from Low to High while SCL remains High.

This informs every slave that the transaction has completed.

The bus then returns to the Idle state.

8. Synchronization

Since every transmission follows this sequence, both the master and the slave remain synchronized throughout the entire communication process.

Reliable communication depends on both devices performing their operations at exactly the correct moments.

The transmitter changes the SDA line only while SCL is LOW (falling edge).

The receiver samples SDA only while SCL is HIGH (rising edge).

This separation guarantees that the transmitted data has sufficient time to stabilize before being sampled.

This timing relationship is maintained throughout every stage of the communication process.

9. System Architecrture

9.1 Top-Level Architecture

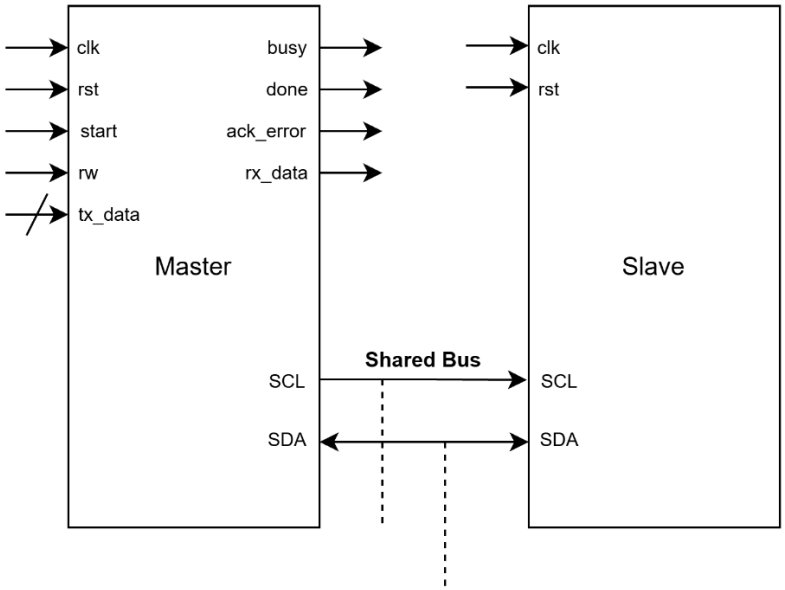


Figure 3- Top-level architecture of the implemented I²C communication system.

9.2 I²C Master

The I²C Master controls all communication over the bus. It generates the serial clock, initiates every transaction, transmits the slave address, performs read and write operations, and generates the START and STOP conditions.

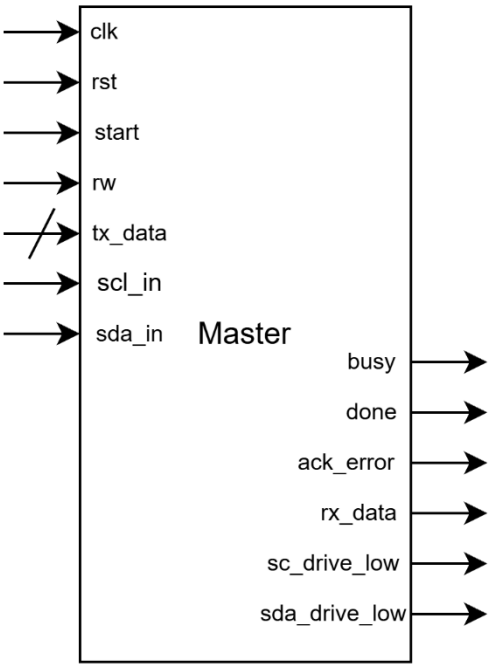


Figure 4 - Master Block Diagram

Port	Direction	Width	Description
clk	Input	1	FPGA system clock.
rst	Input	1	Active High asynchronous reset.
start	Input	1	Begins a new I²C transaction.
rw	Input	1	Selects write (0) or read (1).
tx_data	Input	8	Byte transmitted during write operations.
scl_in	Input	1	Current state of the SCL bus.
sda_in	Input	1	Current state of the SDA bus.
scl_drive_low	Output	1	Drives SCL low or releases it.
sda_drive_low	Output	1	Drives SDA low or releases it.
busy	Output	1	Indicates an active transaction.
done	Output	1	Pulses when the transaction completes.
ack_error	Output	1	Indicates that an ACK was not received.
rx_data	Output	8	Byte received during read operations.

9.3 I²C Slave

The I²C Slave monitors the bus for its assigned address. After a successful address match, it acknowledges the request and either receives data from the master during write operations or transmits previously stored data during read operations.

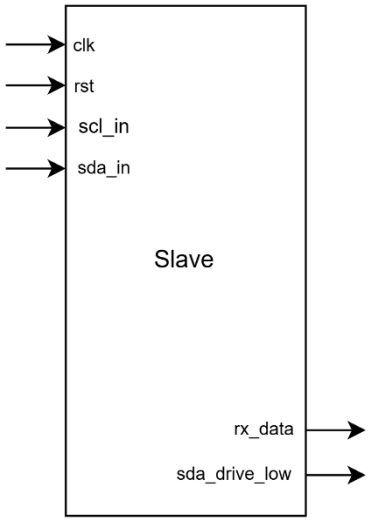


Figure 5- Slave Block Diagram

Port	Direction	Width	Description
clk	Input	1	FPGA system clock.
rst	Input	1	Asynchronous reset.
scl_in	Input	1	Serial clock from the bus.
sda_in	Input	1	Current state of the SDA bus.
sda_drive_low	Output	1	Drives SDA low or releases it.
rx_data	Output	8	Stores the last byte written by the master.

9.4 Clock Divider

The Clock Divider converts the 50 MHz FPGA system clock into a 100 kHz SCL clock for Standard-Mode I²C communication. The generated clock is used by the master to synchronize all data transfers on the bus.

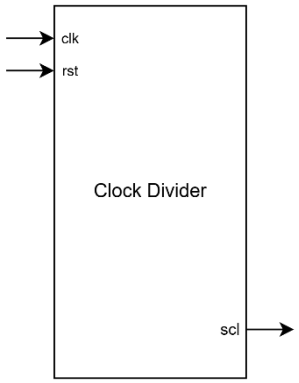


Figure 6 - Clock Divider Block Diagram

Port	Direction	Width	Description
clk	Input	1	FPGA system clock.
rst	Input	1	Asynchronous reset.
scl	Output	1	Generated I²C serial clock.

10. Finite State Machine Design

Both the Master and Slave controllers are implemented as Moore finite state machines (FSMs). The outputs depend only on the current state, while transitions between states occur according to the input conditions and synchronization signals. This approach simplifies the controller design and ensures predictable operation throughout the communication process.

10.1 Master FSM

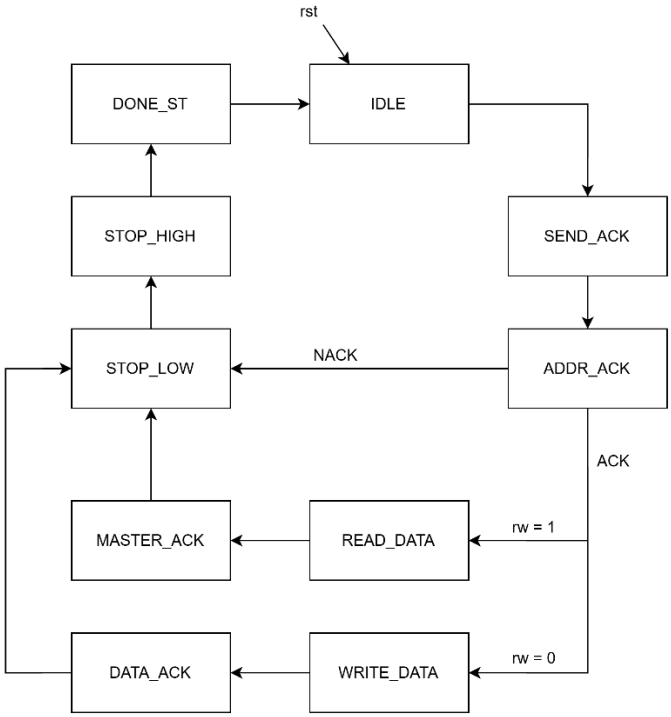


Figure 7- State Diagram for the Master FSM.

10.1.1State Breakdown:

1- IDLE

The master waits for the `start` signal while the bus remains idle. During this state, the SDA and SCL lines are released (HIGH), `busy` is Low, and the controller waits for a new transaction request.

2- START_ST

The master initiates communication by generating the START condition. This is achieved by pulling the SDA line low while the SCL line remains high. At the same time, the slave address and the Read/Write bit are loaded into the shift register in preparation for transmission.

3- SEND_ADDR

The master transmits the 7-bit slave address followed by the Read/Write bit. One bit is transmitted on every falling edge of SCL by shifting the address register toward the SDA output. After all the 8 bits have been transmitted, the master releases the SDA line and proceeds to the address acknowledgement state

4- ADDR_ACK

The master waits for the slave to acknowledge the received address by pulling SDA low during the acknowledgement clock pulse.

If an acknowledgement is received:

The master enters `WRITE_DATA` for a write operation or enters `READ_DATA` for a read operation.

If no acknowledgement is received, an acknowledgement error is generated and the transaction is terminated.

5- WRITE_DATA

During a write transaction, the master loads the transmit byte into the shift register and serially transmits one bit on every falling edge of SCL.

After the complete byte has been transmitted, the master waits for the slave's acknowledgement.

6- DATA_ACK

The master releases the SDA line and waits for the slave to acknowledge successful reception of the transmitted data byte.

If an ACK is received, the communication proceeds to the STOP sequence.

Otherwise, an acknowledgement error is reported before terminating the communication.

7- READ_DATA

During a read transaction, the master releases the SDA line, allowing the slave to drive the bus.

One data bit is sampled on every rising edge of SCL and stored in the receive register until the complete byte has been reconstructed.

After receiving all eight bits, the master enters the `MASTER_ACK` state.

8- MASTER_ACK

After successfully receiving the data byte, the master releases the SDA line during the acknowledgement clock pulse before terminating the communication.

Once the acknowledgement phase is complete, the FSM proceeds to the STOP sequence.

9- STOP_LOW

The master prepares the STOP condition by driving SDA low while SCL is low.

10- STOP_HIGH

When SCL becomes high, the master releases SDA, producing the STOP condition and ending the communication.

11- DONE_ST

The done signal is asserted for one system clock cycle to indicate that the transaction has been completed successfully.

The FSM then returns to the IDLE state.

10.2 Slave FSM

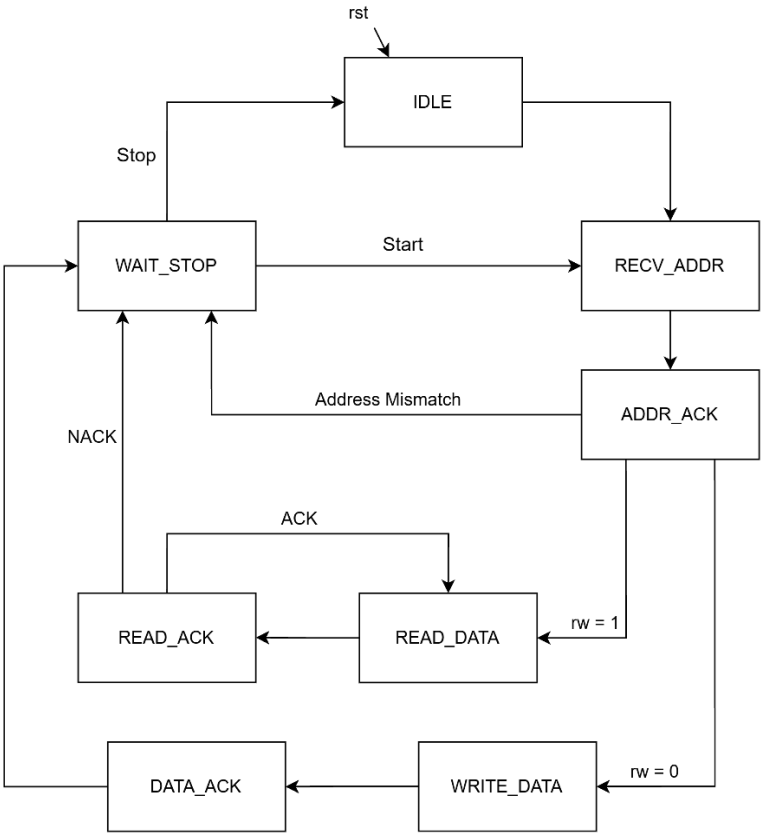


Figure 8- State Diagram for the Slave FSM.

10.2.1 State Breakdown

1- IDLE

The slave remains in the IDLE state while monitoring the SDA and SCL lines for a valid `START` condition.

During this state, the SDA line is released (High), and no data is transmitted or received. Once a `START` condition is detected, the slave clears the shift register, initializes the bit counter, and proceeds to the `RECV_ADDR` state.

2- RECV_ADDR

The slave receives the 7-bit slave address followed by the Read/Write bit from the master. One bit is sampled on every rising edge of the SCL signal and stored in the shift register. After all 8 bits have been received, the Read/Write bit is stored, and the FSM proceeds to the `ADDR_ACK` state.

3- ADDR_ACK

The slave compares the received address with its predefined address.

If the addresses do not match, the slave ignores the remainder of the transaction and waits for a `STOP` condition before returning to the `IDLE` state.

If the addresses match, the slave generates an ACK by pulling the SDA line low during the acknowledgement clock pulse.

After acknowledging the address:

- If the Read/Write bit indicates a `Write` operation, the slave enters the `WRITE_DATA` state.
- If the Read/Write bit indicates a Read operation, the transmit register is loaded into the shift register and the slave enters the `READ_DATA` state.

4- WRITE_DATA

During a Write transaction, the slave receives one data byte transmitted by the master. Each bit is sampled on the rising edge of SCL and shifted into the receive register.

After receiving all 8 bits, the slave proceeds to the `DATA_ACK` state.

5- DATA_ACK

The slave acknowledges successful reception of the data byte by pulling the SDA line Low during the acknowledgement clock pulse.

Once the acknowledgement has been transmitted, the received byte is copied into both the internal data register and the receive output register (rx_data).

The slave then enters the `WAIT_STOP` state.

6- READ_DATA

During a read transaction, the slave becomes the transmitter. The previously stored data byte is loaded into the shift register and transmitted serially over the SDA line.

One bit is transmitted on every falling edge of the SCL signal by shifting the register toward the SDA output.

After transmitting all 8 bits, the slave proceeds to the `READ_ACK` state.

7- READ_ACK

After transmitting one data byte, the slave releases the SDA line and waits for the master's acknowledgement.

If the master transmits an ACK (logic 0), the slave reloads the stored data into the shift register and begins transmitting another byte by returning to the `READ_DATA` state.

If the master transmits a NACK (logic 1), the slave assumes that no additional data is requested and proceeds to the `WAIT_STOP` state.

8- WAIT_STOP

The slave releases the SDA line and waits for the master to generate a `STOP` condition, indicating the end of the communication.

If a STOP condition is detected, the FSM returns to the `IDLE` state.

If a repeated `START` condition is detected instead of a `STOP` condition, the slave immediately returns to the `RECV_ADDR` state to begin processing the next transaction without first returning to `IDLE`.

11. Verification Methodology

The functionality of the implemented I²C communication system was verified using a self-checking Verilog testbench. The testbench instantiates the top-level module, generates the FPGA system clock and reset signals, and automatically performs a sequence of write and read transactions while monitoring the status outputs.

To simplify verification, the testbench uses reusable tasks for the reset, write, and read operations. Each task initiates a complete I²C transaction and waits for the busy and done signals before proceeding to the next test case. At the end of every transaction, the testbench checks the communication status and displays the transaction result.

11.1 Test Cases

The following test cases were performed to verify the functionality of the design.

Test Case	Description	Expected Result
Reset	Applies an asynchronous reset before starting communication.	All internal registers are initialized and both master and slave enter the IDLE state.
Single Write	Writes the value `0xA5` from the master to the slave.	The slave acknowledges the transfer and stores `0xA5` in its receive register.
Multiple Write	Writes `0x00`, `0xFF`, `0x55`, and `0xAA` sequentially.	Each transaction completes successfully and the slave stores the transmitted byte.
Read Operation	The master performs a read transaction.	The slave transmits its stored data and the master correctly reconstructs the received byte.
Random Write	Ten randomly generated bytes are written to the slave.	Every transaction completes successfully without acknowledgement errors.

11.2 Self-Checking Mechanism

The testbench automatically verifies the correctness of each transaction.

For write operations, the testbench waits for the done signal before reporting successful completion. For read operations, the received byte is compared against the expected value. If the received data does not match the expected value, an error message is displayed. The `ack_error` signal is also monitored throughout all transactions to detect missing acknowledgements.

Additionally, whenever a transaction is completed, the values of busy, done, `ack_error`, `rx_data`, and `slave_rx_data` are displayed to simplify debugging and waveform analysis.

11.3 Waveforms Snippet

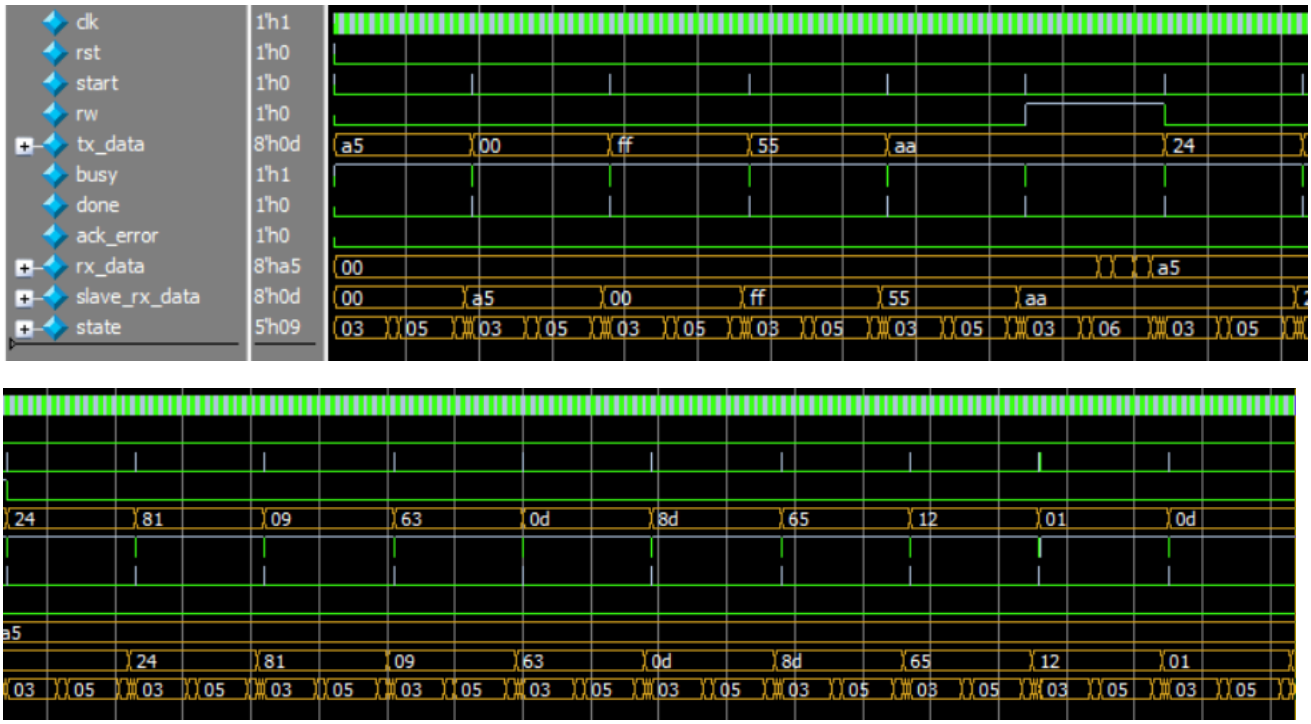


Figure 9- QuestaSim Simulation Waveforms

12. Simulation Results

12.1 QuestaSim Simulation Output

```
# T=3040110 state=9 bit=0 scl=0 rise=0 fall=0 shift=00
# T=3040130 state=9 bit=0 scl=0 rise=0 fall=0 shift=00
# T=3040150 state=9 bit=0 scl=0 rise=0 fall=0 shift=00
# T=3040170 state=9 bit=0 scl=0 rise=0 fall=0 shift=00
# T=3040190 state=9 bit=0 scl=0 rise=0 fall=0 shift=00
# T=3040210 state=9 bit=0 scl=0 rise=0 fall=0 shift=00
# T=3040230 state=9 bit=0 scl=1 rise=1 fall=0 shift=00
# T=3040250 state=10 bit=0 scl=1 rise=0 fall=0 shift=00
# Time=3040250 Done Busy=0 ACK_Error=0 Rx=a5 Slave_Rx=0d
# Write success DATA=0d
# T=3040270 state=0 bit=0 scl=1 rise=0 fall=0 shift=00
# All tests done
# T=3040290 state=0 bit=0 scl=1 rise=0 fall=0 shift=00
```

Figure 10- Self-Checking Testbench Simulation Output

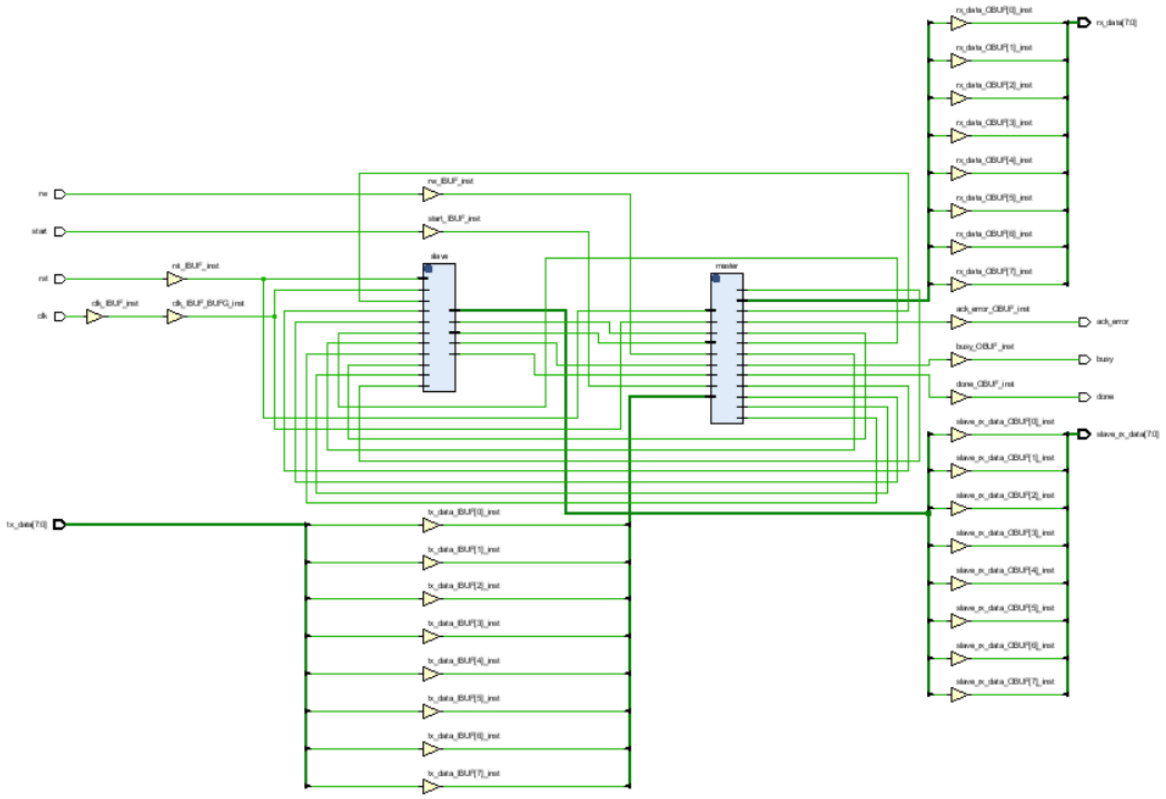
12.2 Linting

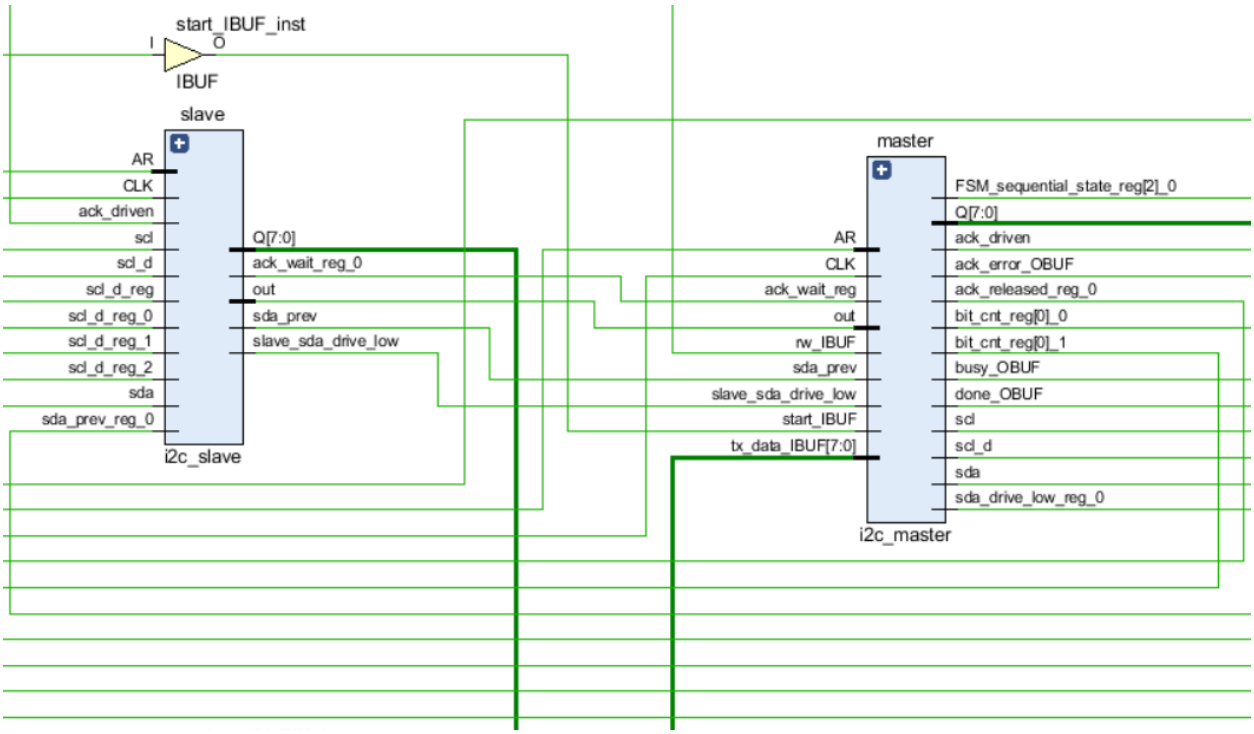
Severity	Status	Check	Alias	Message
⚠	?	seq_block_has_duplicate…		Signal is assigned more than once in a sequential block. Signal sda_dri.
⚠	?	seq_block_has_duplicate…		Signal is assigned more than once in a sequential block. Signal sda_dri.
⚠	?	seq_block_has_duplicate…		Signal is assigned more than once in a sequential block. Signal state, ..
i	?	async_reset_active_high		Asynchronous reset is active high. Reset rst, Module i2c_clk_div, File D
i M	?	async_reset_active_high		Asynchronous reset is active high. Reset rst, Module i2c_master, File ..
i M	?	async_reset_active_high		Asynchronous reset is active high. Reset rst, Module i2c_slave, File D:/
i	?	flop_redundant		Redundant flip-flops found. Flops count 2, First flop slave.data_reg, Mo
i	?	flop_redundant		Redundant flip-flops found. Flops count 2, First flop master.scl_d, Mod.
i	?	parameter_name_duplicate		Same parameter name is used in more than one module. Parameter C.
i	?	parameter_name_duplicate		Same parameter name is used in more than one module. Parameter I2
i	?	parameter_name_duplicate		Same parameter name is used in more than one module. Parameter S.
i	?	parameter_name_duplicate		Same parameter name is used in more than one module. Parameter ID
i	?	parameter_name_duplicate		Same parameter name is used in more than one module. Parameter A.
i	?	parameter_name_duplicate		Same parameter name is used in more than one module. Parameter W
i	?	parameter_name_duplicate		Same parameter name is used in more than one module. Parameter R.
i	?	parameter_name_duplicate		Same parameter name is used in more than one module. Parameter D.

Figure 11- Linting Report Showing No Errors

12.3 Synthesized Design

12.3.1 Schematic





12.2.1 Utilization

Name	Slice LUTs (20800)	Slice Registers (41600)	Bonded IOB (106)	BUFGCTRL (32)
i2c_top	136	68	31	1
master (i2c_master)	71	40	0	0
slave (i2c_slave)	65	28	0	0

13. Summary

This project presented the design and implementation of a complete I²C communication system using Verilog HDL. The system consists of an I²C Master, an I²C Slave, and a clock divider communicating over the shared SDA and SCL bus lines. The operation of the protocol, including START and STOP conditions, addressing, acknowledgements, and both read and write transactions, was discussed alongside the finite state machines governing the master and slave behavior.

14. Conclusion

The implemented design successfully demonstrates reliable I²C communication between a master and a slave device. Simulation results and the self-checking testbench verified the correct operation of write and read transactions, address recognition, data transfer, and acknowledgement handling. The synthesized design and linting results further confirm that the implementation is functionally correct and suitable for FPGA realization.